A Base–Density / Sampler Inconsistency in the *Lévy-Flow* Variance Gamma Implementation

Code Review Note

October 23, 2026

Abstract

The *Lévy-Flow* codebase (`levy_flows/distributions/variance_gamma.py`) implements the Variance Gamma (VG) base distribution described in *Lévy-Flow Models: Heavy-Tail-Aware Normalizing Flows for Financial Risk Management* (Drissi, 2026) using a hand-rolled, piecewise approximation of the modified Bessel function $K_\nu(z)$ rather than the exact function the paper's own density formula (Eq. 2 of that paper) requires. This note (i) recaps the relevant theory from the paper, (ii) shows analytically and numerically that the approximation used in `_log_bessel_k` departs from the true $K_\nu(z)$ by up to $\Delta(\log K) \approx 1.8$ nats (a $\approx 6\times$ multiplicative density error) exactly in the argument range that standardized financial return data populates, (iii) shows that this is compounded by a second, independent bug: `log_prob` and `rsample` are not the density and sampler of the same distribution, so the fitted model does not define a coherent probability law, and (iv) specifies a drop-in fix using `scipy.special.kv` with an exact z-gradient and a finite-difference order-gradient, together with a verification protocol against the empirical results already observed (low training loss, KS $= 0.256$, $p \approx 0$).

1 Recap of the Paper's Theory

1.1 The VG base density

The paper (Section 2.1.1, Eq. 2) defines the Variance Gamma density as

$$\begin{aligned} p_{VG}(x;\mu,\sigma,\theta,\nu) &= \frac{2\,\exp\big(\gamma(x-\mu)\big)}{\sigma\sqrt{2\pi}\,\nu^{1/\nu}\,\Gamma(1/\nu)} \left(\frac{|x-\mu|}{\beta^{-1}}\right)^{1/\nu-1/2} K_{1/\nu-1/2}\big(\beta\,|x-\mu|\big), \\ \beta &= \frac{\sqrt{2\sigma^2/\nu+\theta^2}}{\sigma^2}, \quad \gamma = \frac{\theta}{\sigma^2}, \end{aligned} \tag{1}$$

with K_ν the modified Bessel function of the second kind. Equation (1) is *exact*: K_ν here is a specific, well-defined special function, not a family of approximations. Any implementation that replaces K_ν with a function $\widetilde{K}_\nu \neq K_\nu$ is, by construction, no longer evaluating p_{VG} — it is evaluating some other function $\widetilde{p}_{VG} \neq p_{VG}$.

The paper also specifies the reparameterized sampler (Eq. 3, Section 4.3.1):

$$X = \mu + \theta G + \sigma\sqrt{G}\,Z, \qquad G \sim \text{Gamma}(1/\nu,\, 1/\nu), \quad Z \sim \mathcal{N}(0,1). \tag{2}$$

Equations (1) and (2) are two representations of *the same law* $\mathcal{L}_{VG}(\mu,\sigma,\theta,\nu)$: one via its density, one via a mixing construction. An implementation is only internally consistent if `log_prob` evaluates (1) and `rsample` draws from (2) *for the same distribution*.

1.2 The flow log-likelihood

Section 4.1–4.2 (Eq. 8) gives the model log-likelihood for the composed Lévy-Flow $X = f_\theta(Z)$, $Z \sim p_{VG}(\cdot;\phi)$:

$$\log p_X(x;\theta,\phi) = \log p_{VG}\big(f_\theta^{-1}(x);\phi\big) + \log\left|\det J_{f_\theta^{-1}}(x)\right|. \tag{3}$$

Training minimizes $\mathcal{L}(\theta,\phi) = -\frac{1}{N}\sum_i \log p_X(x_i;\theta,\phi)$ using (3). Whatever function stands in for p_{VG} inside this objective is the function θ is actually optimized against.

1.3 Tail preservation (Proposition 1)

Proposition 1 of the paper states that for an NSF with identity tails outside $[-B,B]$, $p_X(x) = p_Z(x)$ for $|x| > B$, so *whatever* base law p_Z is used, its tail shape is preserved exactly beyond the spline region. This is correct as stated — but it is a statement about *whichever density function is actually used as p_Z inside the training objective*. If that function is not p_{VG} but some approximation $\widetilde{p}_{VG}$, then Proposition 1 guarantees only that $\widetilde{p}_{VG}$'s tails are preserved, not the true VG tails the paper's abstract and title advertise.

1.4 The paper's own stated numerical method

Critically, Section 4.3.2 of the paper is explicit about how $K_\nu(z)$ should be evaluated for numerical stability:

> *"For large z, we use the asymptotic expansion:* $\log K_\nu(z) \approx \frac{1}{2}\log\frac{\pi}{2z} - z,\ z \gg 1$. *For moderate z, we use the exponentially-scaled Bessel function $K_\nu^{(e)}(z) = e^z K_\nu(z)$ to avoid overflow."*

Note what this *is* and *is not*: it is a numerically-stable *re-scaling* of the exact function ($K_\nu^{(e)}$ is exactly $e^z K_\nu(z)$, recoverable exactly as $K_\nu(z) = e^{-z}K_\nu^{(e)}(z)$ with no approximation error introduced for moderate z); it is *not* a license to replace K_ν with a structurally different closed-form approximation in that regime. As shown below, the code does the latter.

2 The Implementation

2.1 What `_log_bessel_k` actually computes

`levy_flows/distributions/variance_gamma.py` defines a function `_log_bessel_k(v, z)` intended to return $\log K_v(z)$ using "pure PyTorch" for GPU-compatibility and autograd. It is built from three pieces:

(a) Large-z asymptotic series.

$$\begin{aligned}\log K_v^{\text{asym}}(z) = {} & \underbrace{\tfrac{1}{2}\log\tfrac{\pi}{2z} - z}_{\text{leading term (paper's stated formula)}} \\ & + \log\left(1 + \frac{4v^2-1}{8z} + \frac{(4v^2-1)(4v^2-9)}{2!(8z)^2} + \frac{(4v^2-1)(4v^2-9)(4v^2-25)}{3!(8z)^3}\right).\end{aligned} \tag{4}$$

This is the standard asymptotic expansion of $K_v(z)$ for $z \to \infty$ truncated at three correction terms — valid, but *only* as $z \to \infty$; the series is asymptotic, not convergent, and the truncation error is $O\big((4v^2-1)(4v^2-9)(4v^2-25)(4v^2-49)/z^4\big)$, which is not controlled for $z = O(1)$.

(b) Small-z leading term.

$$\log K_v^{\text{small}}(z) = \Gamma_{\log}(|v|) + (|v|-1)\log 2 - |v|\log z, \qquad \Gamma_{\log} \equiv \log\Gamma(\cdot). \tag{5}$$

This is the leading term of $K_v(z) \sim \frac{1}{2}\Gamma(v)(z/2)^{-v}$ as $z \to 0^+$ for fixed $v > 0$ — again valid only in that limit, and it drops the full hypergeometric-type series entirely.

(c) Sigmoid blend.

$$\log \widetilde{K}_v(z) = w(z)\ \log K_v^{\text{asym}}(z) + \big(1-w(z)\big)\ \log K_v^{\text{small}}(z), \qquad w(z) = \sigma\big(2(z-\tau(v))\big), \quad \tau(v) = 2+2|v|. \tag{6}$$

Equation (6) is an *ad hoc interpolant*, not a mathematical identity. There is no result in the special-functions literature stating that a sigmoid-weighted convex combination of the two limiting expansions of K_v approximates K_v well in between — and, as Section 3 shows, it does not. This is the crux of the bug: *the function $\widetilde{K}_v$ actually computed is not K_v, is not claimed by any theorem to be close to K_v away from the two limits, and does not correspond to the exponentially-scaled-Bessel method the paper itself specifies.*

2.2 Where the error lands

For standardized data ($\sigma \approx 1$), the Bessel argument in (1) is $z = \beta|x-\mu|$ with $\beta = O(1)$. Since x is standardized, the empirical mass of $|x-\mu|$ is concentrated on $[0, 3]$ (verified against the paper's own reported excess-kurtosis, Section 5.2.1: 10.64), placing z predominantly in $[0.1, 5]$ — i.e., *directly inside the blend zone* of (6), not the tails where either limiting expansion is trustworthy.

3 Numerical Verification

Equation (6) was reimplemented verbatim in NumPy and compared against the exact reference value from `scipy.special.kv` at $\nu = 0.6$, $\theta = -0.1$, $\sigma = 1$ (giving Bessel order $v = 1/\nu - 1/2 = 1.167$, blend threshold $\tau(v) = 4.33$):

z	exact $\log K_v(z)$	code's $\log \widetilde{K}_v(z)$	$\lvert\Delta\rvert$ (nats)
0.05	3.5329	3.5364	0.0035
0.10	2.7176	2.7275	0.0099
0.30	1.3819	1.4454	0.0634
0.50	0.7034	0.8492	0.1459
1.00	−0.3823	0.0400	**0.4223**
1.50	−1.1900	−0.4351	**0.7549**
2.00	−1.8938	−0.7786	**1.1152**
3.00	−3.1630	−1.3660	**1.7970**
5.00	−5.4774	−4.7179	0.7595
8.00	−8.7484	−8.7443	0.0042

Table 1: Error in $\log K_v(z)$ introduced by `_log_bessel_k`, at the order v implied by the model's own fitted-region parameters. Errors peak at $|\Delta| \approx 1.8$ nats near $z = 3$ — a multiplicative density error of $e^{1.8} \approx 6.0\times$ — precisely inside the data-populated range identified above.

The error is not a symmetric, mean-zero perturbation: it is a *systematic over-estimate* of $\log K_v(z)$ throughout the blend zone (the code's value exceeds the true value at every listed z from

0.05 to 5.0). A systematic bias in the log-density directly biases the fitted $(\mu, \sigma, \theta, \nu)$ and flow parameters, and inflates or deflates the apparent likelihood independent of how well the model actually matches the data's shape.

4 Diagnosis: Two Compounding Failures

4.1 Failure 1 — $\widetilde{p}_{VG} \neq p_{VG}$

Because (6) replaces K_v with $\widetilde{K}_v$, the function actually computed by `VarianceGamma.log_prob` is not (1) but

$$\widetilde{p}_{VG}(x) = p_{VG}(x) \cdot \exp\big(\log \widetilde{K}_v(z) - \log K_v(z)\big), \qquad z = \beta|x - \mu|, \tag{7}$$

i.e. p_{VG} multiplied by a z-dependent, non-constant distortion factor (Table 1). Since the distortion is not constant in x, $\widetilde{p}_{VG}$ is not merely a rescaled version of p_{VG}: it is a *differently shaped* function, and there is no guarantee it integrates to 1.

4.2 Failure 2 — `log_prob` and `rsample` score different distributions

Examine the two methods side by side:

```
def log_prob(self, x):
    ...
    log_bessel = _log_bessel_k(bessel_order, bessel_arg) # uses widetilde-K
    log_p = log_norm + log_power + log_bessel + exp_term
    return log_p.sum(dim=-1)

def rsample(self, num_samples):
    G = Gamma(1/nu, 1/nu).rsample((num_samples,)) # EXACT mixing draw
    Z = torch.randn(...)
    return mu + theta * G + sigma * torch.sqrt(G) * Z # EXACT VG law, Eq. (2)
```

`rsample` is exact: it draws from $\mathcal{L}_{VG}(\mu, \sigma, \theta, \nu)$ as defined by (2), with no Bessel function involved at all. `log_prob` scores points under $\widetilde{p}_{VG} \neq p_{VG}$. These are *not the density and sampler of the same object*. Formally, letting $\theta^\star$ denote the flow parameters at the end of training:

$$\theta^\star = \arg\min_\theta \; -\tfrac{1}{N} \sum_i \Big[\log \widetilde{p}_{VG}\big(f_\theta^{-1}(x_i)\big) + \log \big| \det J_{f_\theta^{-1}}(x_i) \big| \Big] \quad \text{(what training actually optimizes)} \tag{8}$$

$$X^{\text{sampled}} = f_{\theta^\star}^{-1}(Z), \quad Z \sim p_{VG}(\cdot) \text{ (exact, via } \texttt{rsample}\text{)} \qquad \text{(what } \texttt{sample()} \text{ returns)} \tag{9}$$

$f_{\theta^\star}$ was tuned in (8) to map the data distribution close to $\widetilde{p}_{VG}$, *not* p_{VG}. Pushing *true* p_{VG} samples back through $f_{\theta^\star}^{-1}$ in (9) therefore does not, in general, reproduce the data distribution — even if (8) were driven to its exact minimum. This is a structural inconsistency, not an optimization or convergence issue, and no amount of additional training fixes it.

4.3 Why this matches the observed experiment exactly

This diagnosis predicts precisely the qualitative pattern seen in the `financial_experiment.py` run:

- **Low training loss** (0.3936, lowest of all four models tested): training minimizes (8) against $\widetilde{p}_{VG}$, and since $\widetilde{K}_v$ systematically *over*-states $\log K_v$ (Table 1), $\widetilde{p}_{VG}$ can assign higher likelihood than the true density would — the loss is optimized against an artificially favorable objective.
- **Terrible KS statistic** (0.256, $p \approx 0$, second-worst of the four models): the KS test compares `sample()` output (9) — generated from the *true* p_{VG} pushed through a flow tuned for the *wrong* $\widetilde{p}_{VG}$ — against the real data. Section 4.2 shows this comparison has no reason to succeed.
- **Proposition 1 does not deliver what it promises here**: the tail-preservation guarantee correctly preserves $\widetilde{p}_{VG}$'s tails beyond $\pm B$, but $\widetilde{p}_{VG}$'s tails are not the VG tails of (1) the paper's central claim rests on (Table 1 shows the distortion persists out to $z \approx 5$ before the two asymptotic regimes agree).

5 Fix

Replace $\widetilde{K}_v$ with the exact function, using `scipy.special.kv` inside a custom `torch.autograd.Function` so gradients still flow to z (equivalently, to x, and hence to the flow parameters θ) and to v (equivalently, to ν, if ν is learnable).

5.1 Exact z-gradient

The modified Bessel functions satisfy the standard recurrence

$$\frac{d}{dz}K_v(z) = -\tfrac{1}{2}\big(K_{v-1}(z) + K_{v+1}(z)\big), \tag{10}$$

which is exact for all real v and $z > 0$ (Abramowitz & Stegun 9.6.26). This requires two extra `scipy.special.kv` evaluations per forward pass (at orders $v - 1$ and $v + 1$) and introduces no approximation error.

5.2 Order gradient (if ν, and hence $v = 1/\nu - 1/2$, is learnable)

$K_v(z)$ has no simple closed form for $\partial K_v/\partial v$ in general. A central finite difference is used instead:

$$\frac{\partial K_v(z)}{\partial v} \approx \frac{K_{v+h}(z) - K_{v-h}(z)}{2h}, \qquad h = 10^{-4}. \tag{11}$$

Because v is a scalar shared across the whole batch (one ν per base distribution, not per-sample), (11) costs only two additional scalar-broadcast `scipy.special.kv` evaluations per forward pass, independent of batch size.

5.3 Reference implementation

```
class BesselK(torch.autograd.Function):
    @staticmethod
    def forward(ctx, z, v, fd_eps=1e-4):
        z_np = z.detach().cpu().numpy()
        v_val = float(v.detach().cpu())
        k_v = scipy_kv(v_val, z_np)
```

```
        k_vm1 = scipy_kv(v_val - 1.0, z_np)
        k_vp1 = scipy_kv(v_val + 1.0, z_np)
        dK_dv = (scipy_kv(v_val + fd_eps, z_np)
                 - scipy_kv(v_val - fd_eps, z_np)) / (2 * fd_eps)
        ctx.save_for_backward(
            torch.as_tensor(k_vm1, dtype=z.dtype, device=z.device),
            torch.as_tensor(k_vp1, dtype=z.dtype, device=z.device),
            torch.as_tensor(dK_dv, dtype=z.dtype, device=z.device))
        ctx.v_shape, ctx.v_requires_grad = v.shape, v.requires_grad
        return torch.as_tensor(k_v, dtype=z.dtype, device=z.device)

    @staticmethod
    def backward(ctx, grad_output):
        k_vm1, k_vp1, dK_dv = ctx.saved_tensors
        grad_z = grad_output * (-0.5 * (k_vm1 + k_vp1)) # Eq. (10)
        grad_v = (grad_output * dK_dv).sum().reshape(ctx.v_shape) \
                 if ctx.v_requires_grad else None # Eq. (11)
        return grad_z, grad_v, None
```

This was verified against a from-scratch numeric derivative of the *full* VG log-density (not just K_v in isolation) with respect to ν: autograd gave $\partial(\sum \log p_{VG})/\partial\nu = -0.7684$ against a central-difference numeric reference of -0.7672 — agreement to 3 significant figures, confirming both (10) and (11) are wired correctly end-to-end through the softplus reparameterization used to keep $\nu > 0$ during optimization.

5.4 Relation to the paper's stated method

Note this fix does *not* need the exponentially-scaled trick the paper describes (Section 4.3.2, $K_v^{(e)}(z) = e^z K_v(z)$): for the z-range relevant to standardized daily-return data ($z \lesssim 10$), `scipy.special.kv` does not overflow, so the direct, exact value can be used. If z were expected to range much higher (e.g. heavily non-standardized raw price levels), `scipy.special.kve` (the already-implemented exponentially-scaled variant) combined with an explicit $-z$ correction in log-space would reproduce the paper's stated method exactly, still without the approximation error of (6).

6 Verification Protocol

After patching `_log_bessel_k` (or replacing it with Section 5's `BesselK`), re-run `financial_experiment.py` Experiment 1 and check:

1. **KS statistic** for the VG model should drop substantially below its current 0.256 — ideally into the same range as the paper's other models with passing/borderline KS tests (Student-t: 0.045), since `log_prob` and `sample()` will now score/generate the same distribution.

2. **Training loss will likely rise** from 0.3936: this is expected and correct. The current loss is artificially low precisely because it is computed against $\widetilde{p}_{VG}$ (Section 4.2); a higher, honest loss against the true p_{VG} is the sign the fix worked, not a regression.

3. **VaR/ES backtests** (Experiment 2) and crisis-period tail comparisons (Experiment 3) should be re-run, since both currently draw samples via the same inconsistent `sample()` path.

4. Cross-check the new loss against an independent VG-only (no-flow) ablation fit with the corrected density, the same check that originally surfaced this class of bug in a separate re-implementation (VG-only should not, in a correct implementation, beat a properly-trained VG-flow on validation NLL).